

Servidores MCP frente a herramientas ad hoc

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 10 de 12 · Notas en español (es-ES)

Las herramientas ad hoc viven dentro de una sola app de IA: escribís `get_current_date_time`, registráis el esquema y entregáis ese cuaderno. El **Model Context Protocol (MCP)** es la misma idea desplazada a un *servidor* que otros clientes pueden compartir. La frase de Vijay: escribid los métodos una vez y consumidlos desde Claude Code, Cowork, GitHub Copilot o vuestro propio cliente Messages.

El problema de integración N×N

Imaginad tres fuentes internas (una base de datos, un repositorio de documentos, un conjunto de APIs) y tres apps de IA que las necesitan. El cableado punto a punto son nueve integraciones. Cambiáis una columna y parcheáis tres apps. MCP inserta dos roles para que las apps dejen de importar SDK de proveedor directamente:

- **Servidor MCP** — se conecta a la fuente de datos y *expone herramientas* (y recursos relacionados) por el protocolo.
- **Cliente MCP** — vive en la aplicación de IA, lista esas herramientas y deja que Claude elija entre ellas como con cualquier otro esquema.

Cuando cambia el contrato de la base de datos, actualizáis el servidor. Los clientes conservan la misma configuración de conexión. El aislamiento es el objetivo, no un LLM nuevo.

PROTOCOLO ABIERTO

MCP no es magia solo de Claude. Es un protocolo abierto sobre cómo las aplicaciones dan contexto a los modelos. Confirmad la documentación vigente en [MCP en docs.claude.com](#): las interfaces de marketplace de la grabación se habrán movido.

De dónde sale el cliente

En los stacks de Claude Code / agentes Anthropic, el formador trató el **cliente MCP como integrado**. No implementáis JSON-RPC a mano. *Configuráis* a qué servidores engancharos. El SDK de Anthropic también trae ayudas de cliente. El trabajo de plataforma suele ser **construir o alojar servidores** para datos propietarios.

Tres formas de servidor en clase

Forma	Quién aloja los datos	Ejemplo de aula
Servidor propietario local	Vosotros, en la misma máquina que el cliente	stdio / IO cuando cliente y servidor comparten anfitrión

Forma	Quién aloja los datos	Ejemplo de aula
Vuestro servidor envolviendo HTTP de terceros	Llamáis a una API pública o interna y reexponéis herramientas MCP	Demo: get todos, get energy prices
Servidor del proveedor	Figma, Google, Playwright, ElevenLabs, ...	MCP de Figma en Claude Cowork; el conector de Gmail también es MCP

Los «conectores» de Cowork en este curso son adjuntos MCP (o de forma MCP). Las herramientas de *código* por defecto que escribisteis en el cuaderno no lo son; son funciones en proceso. El SaaS de terceros es donde MCP es el patrón por defecto hoy.

Marketplaces y registros

No tenéis que inventar cada servidor. La sesión citó marketplaces y registros MCP (marketplace de VS Code, registro MCP de GitHub, catálogo de servidores de Claude). El listado de Playwright, por ejemplo, anunciaba herramientas de tamaño de navegador, cierre y mensajes de consola: un LLM elige entre ellas.

- Preferid el servidor MCP **del propio proveedor** cuando consumáis Figma (o similar), no un clon aleatorio de terceros.
- Si *vosotros* publicáis un servidor, vale cualquier biblioteca que implemente el protocolo (FastMCP, SDK oficiales, envoltorios FastAPI). Las bibliotecas de Anthropic conocen los casos límite; importa más que el protocolo esté completo que la marca.
- Un **registro** se parece más a un listado tipo npm/marketplace que a Swagger OpenAPI. MCP no es «REST con pasos extra»: el descubrimiento, la lista de herramientas y los artefactos de conexión son del protocolo. Podéis empaquetar una imagen Docker (servidor que corréis en local) o un «paquete como servicio» (URL remota más configuración de conexión).

Transportes: IO, socket, HTTP

El servidor FastAPI de demostración expuso cuatro herramientas: sumar dos números, precios de energía, todos y saludo. La misma lista puede alcanzarse por canales distintos:

- **IO / stdio** — cliente y servidor en una máquina (MCP local típico de Claude Code).
- **Socket** — misma red, proceso remoto; el formador lo comparó con un envoltorio tipo WebSocket, pero los sockets pueden montar otros protocolos de aplicación (SMTP, FTP) si estáis tendiendo un puente a esos sistemas.
- **HTTP** — servidor en Internet; aun así no escribís a mano cada campo del cuerpo como en un cliente REST improvisado. El cliente MCP rellena el sobre del protocolo.

El `mcp . j` son de GitHub Copilot en esa demo era «¿dónde está la URL (o el comando) de mi servidor?». El cliente MCP de Copilot listaba entonces las herramientas. Tras recargar la ventana y con el proceso en marcha, aparecieron `add / energy prices / todos`. El mismo servidor, varios consumidores.

Herramienta ad hoc frente a herramienta MCP — contraste de examen

Herramienta en proceso		Herramienta MCP
Dónde se define	El código de vuestra app de IA	Un proceso servidor MCP
Reutilización	Copiar/pegar o una biblioteca compartida	Cualquier cliente MCP que alcance el servidor
Gestión del cambio	Redesplegar cada app	Redesplegar el servidor; los clientes conservan la config
Vista de Claude	Un esquema en el array tools	El mismo tipo de esquema, descubierto vía el cliente

MCP no sustituye a RAG ni a la memoria. Es un patrón de conectores para dar contexto (herramientas y, a menudo, recursos) de forma estándar. La sesión 10 giró entonces hacia RAG como siguiente técnica de contexto — se continúa en el módulo 11.

Documentación oficial

- **MCP**
<https://docs.claude.com/en/docs/agents-and-tools/mcp>
Modelo cliente/servidor para productos Claude y la API.
- **Visión general de tool use**
<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>
El apretón de manos en el que también participan las herramientas MCP.
- **Visión general de Claude Code**
<https://code.claude.com/docs/en/overview>
Donde muchos ingenieros enganchan por primera vez servidores MCP.
- **Soporte de Cowork / Claude desktop**
<https://support.claude.com/>
Conectores (Figma, Gmail) como en sesiones anteriores de Cowork.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.